

Linear Data Structures

Data Structure Course
DGIST

November 1, 2025

Contents

1. Introduction
2. Arrays
3. Linked Lists
4. Stacks
5. Queues
6. Deque & Circular Queue
7. Priority Queue
8. Comparison & Use Cases
9. Summary

Introduction

What Are Linear Data Structures?

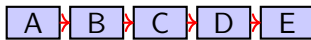
Definition

Linear data structures are structures where elements are arranged **sequentially**, with each element having at most one predecessor and one successor.

Covered Structures:

- Arrays (Static & Dynamic)
- Linked Lists (Singly, Doubly, Circular)
- Stacks (LIFO)
- Queues (FIFO)
- Deques & Circular Queues
- Priority Queues

Linear Arrangement



Sequential Order

Key Characteristic

Arrays

Arrays: The Foundation

Definition

An **array** is a contiguous block of memory storing elements of the same type in sequential memory locations, providing **$O(1)$ indexing**.

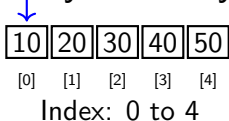
Key Properties:

- Contiguous memory allocation
- Direct access by index
- Fixed or dynamic size
- Cache-friendly (spatial locality)

Memory Address

$$\text{addr}[i] = \text{base} + i \times \text{sizeof}(\text{element})$$

Array in Memory



Static vs Dynamic Arrays

Static Arrays

Fixed size at creation

Pros:

- ✓ No resize overhead
- ✓ Predictable memory
- ✓ Slightly faster

Cons:

- × Cannot grow/shrink
- × Must know size
- × Wasted or insufficient space

Dynamic Arrays

Resizable as needed

Pros:

- ✓ Flexible size
- ✓ $O(1)$ amortized append
- ✓ No size constraints

Cons:

- × Occasional resize ($O(n)$)
- × Extra capacity overhead
- × Unpredictable resizing

Dynamic Array Growth Strategy

How It Works:

1. Start with small capacity (e.g., 4)
2. When full, allocate new array ($2 \times$ size)
3. Copy all elements to new array
4. Free old array

Growth Sequence

Capacity: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \dots$

Amortized $O(1)$

For n insertions:

Total copies = $1 + 2 + 4 + \dots + n = 2n - 1$

Average: $(2n - 1)/n \approx 2 = O(1)$

Python Example

```
1 class DynamicArray:
2     def __init__(self):
3         self._capacity = 4
4         self._size = 0
5         self._data = [None] * 4
6
7     def append(self, item):
8         if self._size == self._capacity:
9             self._resize(2 * self._capacity)
10        self._data[self._size] = item
11        self._size += 1
12
13    def _resize(self, new_cap):
14        new_data = [None] * new_cap
15        for i in range(self._size):
16            new_data[i] = self._data[i]
17        self._data = new_data
18        self._capacity = new_cap
```


Array Performance Summary

Operation	Static	Dynamic	Notes
Access by index	$O(1)$	$O(1)$	Direct memory access
Search (unsorted)	$O(n)$	$O(n)$	Linear scan
Search (sorted)	$O(\log n)$	$O(\log n)$	Binary search
Insert at end	N/A	$O(1)$ amortized	May trigger resize
Insert at middle	$O(n)$	$O(n)$	Shift elements
Delete	$O(n)$	$O(n)$	Shift elements
Memory usage	Exact	Extra capacity	25-50% overhead

When to Use Arrays

- Need fast random access by index
- Size is known or changes infrequently
- Mostly reading data, few insertions/deletions

Linked Lists

Introduction to Linked Lists

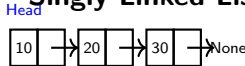
Definition

A **linked list** is a linear structure where elements (nodes) are stored non-contiguously in memory. Each node contains data and a reference (pointer) to the next node.

Key Differences from Arrays:

- × No random access (must traverse)
- ✓ Dynamic size (no preallocation)
- ✓ $O(1)$ insert/delete at known position
- × Extra memory for pointers
- × Poor cache locality

Singly Linked List



Types of Linked Lists

Singly Linked

Each node points to next only



Memory: 1 pointer/node
Traverse: Forward only

Doubly Linked

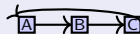
Pointers to next & prev



Memory: 2 pointers/node
Traverse: Both directions

Circular

Last points to first



Forms a circle
No NULL end

Feature	Singly	Doubly	Circular
Memory/node	1 pointer	2 pointers	1-2 pointers
Forward traverse	✓	✓	✓
Backward traverse	×	✓	× (singly)
Insert at beginning	O(1)	O(1)	O(1)
Insert at end	O(n) or O(1)*	O(1)	O(n) or O(1)*

Singly Linked List Implementation

Node Class

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
```

Insert at End

```
1 def insert_at_end(self, data):
2     new_node = Node(data)
3     if not self.head:
4         self.head = new_node
5     else:
6         current = self.head
7         while current.next: # O(n)
8             current = current.next
9         current.next = new_node
10    self.size += 1
```

Insert at Beginning

```
1 def insert_at_beginning(self, data):
2     new_node = Node(data)
3     new_node.next = self.head
4     self.head = new_node
5     self.size += 1
6     # O(1) - just update head
```

Delete Node

```
1 def delete(self, data):
2     if self.head.data == data:
3         self.head = self.head.next
4         return True
5
6     current = self.head
7     while current.next:
8         if current.next.data == data:
9             current.next = current.next.next
10            return True
```

Linked List vs Array

Operation	Array	Linked List
Access by index	$O(1)$	$O(n)$
Search	$O(n)$	$O(n)$
Insert at beginning	$O(n)$	$O(1)$
Insert at end	$O(1)$ amortized	$O(n)$ or $O(1)^*$
Insert at middle	$O(n)$	$O(1)^{**}$
Delete at beginning	$O(n)$	$O(1)$
Delete at middle	$O(n)$	$O(1)^{**}$
Memory overhead	Low	High (pointers)
Cache performance	Excellent	Poor

* $O(1)$ with tail pointer, **If position is known

Use Arrays When

- Need random access
- Size known/stable

Use Linked Lists When

- Frequent insertions/deletions
- Size unknown/dynamic

Stacks

Stacks: LIFO Principle

Definition

A **stack** is a linear data structure following **LIFO** (Last In, First Out). The last element added is the first one removed, like a stack of plates.

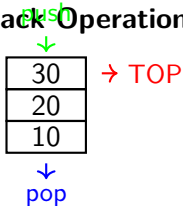
Core Operations (All $O(1)$):

- `push(item)`: Add to top
- `pop()`: Remove from top
- `peek()`: View top without removing
- `is_empty()`: Check if empty
- `size()`: Get element count

Key Restriction

Only the **top** element is accessible. This restriction is intentional and enables many

Stack Operations



Stack Implementations

Array-Based Stack

```
1 class ArrayStack:
2     def __init__(self):
3         self._data = []
4
5     def push(self, item):
6         self._data.append(item) # O(1)
7
8     def pop(self):
9         if self.is_empty():
10             raise IndexError("empty")
11         return self._data.pop() # O(1)
12
13     def peek(self):
14         if self.is_empty():
15             raise IndexError("empty")
16         return self._data[-1] # O(1)
17
18     def is_empty(self):
19         return len(self._data) == 0
20
21     def size(self):
22         return len(self._data)
```

Linked List-Based Stack

```
1 class LinkedStack:
2     def __init__(self):
3         self._head = None
4         self._size = 0
5
6     def push(self, item):
7         new_node = Node(item)
8         new_node.next = self._head
9         self._head = new_node
10        self._size += 1 # O(1)
11
12    def pop(self):
13        if self.is_empty():
14            raise IndexError("empty")
15        item = self._head.data
16        self._head = self._head.next
17        self._size -= 1
18        return item # O(1)
19
20    def peek(self):
21        if self.is_empty():
22            raise IndexError("empty")
23        return self._head.data
24
25    def is_empty(self):
26        return self._head is None
```

Stack Applications

1. Function Call Stack

```
1 def factorial(n):
2     if n <= 1:
3         return 1
4     return n * factorial(n - 1)
5
6 # Call stack for factorial(3):
7 # factorial(3)
8 #   factorial(2)
9 #     factorial(1) <- TOP
10 #       returns 1
11 #     returns 2
12 #   returns 6
```

2. Balanced Parentheses

```
1 def is_balanced(expr):
2     stack = []
3     pairs = {'(':':')', '[':']', '{':'}'}
4
5     for char in expr:
6         if char in pairs:
```

3. Undo/Redo

```
1 class TextEditor:
2     def __init__(self):
3         self.text = ""
4         self.undo_stack = []
5         self.redo_stack = []
6
7     def write(self, text):
8         self.undo_stack.append(self.text)
9         self.text += text
10        self.redo_stack.clear()
11
12    def undo(self):
13        if self.undo_stack:
14            self.redo_stack.append(
15                self.text)
16            self.text = \
17                self.undo_stack.pop()
18
19    def redo(self):
20        if self.redo_stack:
21            self.undo_stack.append(
22                self.text)
23            self.text = \
24                self.redo_stack.pop()
```

Queues

Queues: FIFO Principle

Definition

A **queue** is a linear data structure following **FIFO** (First In, First Out). The first element added is the first one removed, like a waiting line.

Core Operations (All $O(1)$):

- `enqueue(item)`: Add to rear
- `dequeue()`: Remove from front
- `front()`: View front without removing
- `is_empty()`: Check if empty
- `size()`: Get element count

Queue Operations

FRONT **REAR**
dequeue →

10	20	30	40
----	----	----	----

 ← enqueue

Key Property

Fair processing: first come, first served

Queue Implementations

Naive (Bad!)

```
1 class NaiveQueue:
2     def __init__(self):
3         self._data = []
4
5     def enqueue(self, item):
6         self._data.append(item) # O(1)
7
8     def dequeue(self):
9         return self._data.pop(0) # O(n)!
10        # Shifts all elements left!
```

Using Deque (Good!)

```
1 from collections import deque
2
3 class Queue:
4     def __init__(self):
5         self._data = deque()
6
7     def enqueue(self, item):
8         self._data.append(item) # O(1)
```

Linked List Queue

```
1 class LinkedList:
2     def __init__(self):
3         self._head = None
4         self._tail = None
5         self._size = 0
6
7     def enqueue(self, item):
8         new_node = Node(item)
9         if self._tail is None:
10             self._head = self._tail = \
11                 new_node
12         else:
13             self._tail.next = new_node
14             self._tail = new_node
15         self._size += 1 # O(1)
16
17     def dequeue(self):
18         if self.is_empty():
19             raise IndexError("empty")
20         item = self._head.data
21         self._head = self._head.next
22         if self._head is None:
```

Queue Applications

1. Task Scheduling

```
1 class TaskScheduler:
2     def __init__(self):
3         self.queue = Queue()
4
5     def add_task(self, task):
6         self.queue.enqueue(task)
7
8     def process_next(self):
9         if not self.queue.is_empty():
10             task = self.queue.dequeue()
11             print(f"Processing: {task}")
12
13 # First added = first processed
```

2. Print Queue

```
1 class PrintQueue:
2     def __init__(self):
3         self.queue = Queue()
4
5     def add_document(self, doc):
6         self.queue.enqueue(doc)
7
```

3. BFS Traversal

```
1 def bfs(graph, start):
2     visited = set()
3     queue = Queue()
4     queue.enqueue(start)
5     visited.add(start)
6
7     while not queue.is_empty():
8         node = queue.dequeue()
9         print(node)
10
11         for neighbor in graph[node]:
12             if neighbor not in visited:
13                 visited.add(neighbor)
14                 queue.enqueue(neighbor)
15
16 # Explores level by level
```

Other Uses

- Request handling (web servers)
- Message queues

Deque & Circular Queue

Deque (Double-Ended Queue)

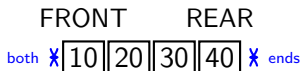
Definition

A **deque** (pronounced "deck") allows insertion and deletion at **both ends** (front and rear). It combines capabilities of stacks and queues.

Operations (All $O(1)$):

- `add_front(item)`
- `add_rear(item)`
- `remove_front()`
- `remove_rear()`
- `front(), rear()`

Deque Structure



Python Implementation

```
1 from collections import deque
```

Use Cases

- Palindrome checking
- Sliding window problems
- Browser history (fast back/forward)

Circular Queue

Definition

A **circular queue** reuses empty spaces when elements are dequeued. The rear wraps around to the beginning when it reaches the end.

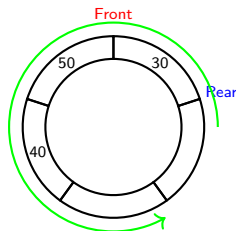
Why Circular?

- Regular queue: empty spaces at front wasted
- Circular queue: reuses freed space
- Fixed capacity (efficient for buffers)

Key Formula

$$\text{rear} = (\text{front} + \text{size}) \% \text{capacity}$$
$$\text{front} = (\text{front} + 1) \% \text{capacity}$$

Circular Queue



Priority Queue

Priority Queue Overview

Definition

A **priority queue** is an ADT where each element has an associated **priority**. Elements with higher priority are dequeued first, regardless of insertion order.

Key Difference:

- Regular queue: FIFO
- Priority queue: Highest priority first

Operations:

- `insert(item, priority): O(log n)`
- `extract_min/max(): O(log n)`
- `peek(): O(1)`

Implementation

Visual Example

Min Priority Queue

Regular:

5 3 8 1 → 5

Priority:

1 3 5 8 → 1

Python Usage

```
1 import heapq
2 heap = []
```

Priority Queue Applications

1. Emergency Room

```
1 class EmergencyRoom:
2     def __init__(self):
3         self.queue = PriorityQueue()
4
5     def admit_patient(self, name, severity):
6         # Lower = more critical
7         self.queue.insert(name, severity)
8
9     def treat_next(self):
10        # Treats most critical first
11        return self.queue.extract_min()
12
13 er = EmergencyRoom()
14 er.admit_patient("Alice", 5) # Minor
15 er.admit_patient("Bob", 1)   # Critical
16 er.treat_next() # Bob (critical)
```

2. Task Scheduler

```
1 class TaskScheduler:
2     def __init__(self):
3         self.pq = PriorityQueue()
```

3. Dijkstra's Algorithm

```
1 def dijkstra(graph, start):
2     distances = {node: float('inf')}
3     for node in graph:
4         distances[node] = 0
5     pq = PriorityQueue()
6     pq.insert(start, 0)
7
8     while not pq.is_empty():
9         current = pq.extract_min()
10
11         for neighbor, weight in \
12             graph[current]:
13             dist = distances[current] + \
14                 weight
15             if dist < distances[neighbor]:
16                 distances[neighbor] = dist
17                 pq.insert(neighbor, dist)
18
19     return distances
```

Other Uses

- Event-driven simulation

Comparison & Use Cases

Performance Summary

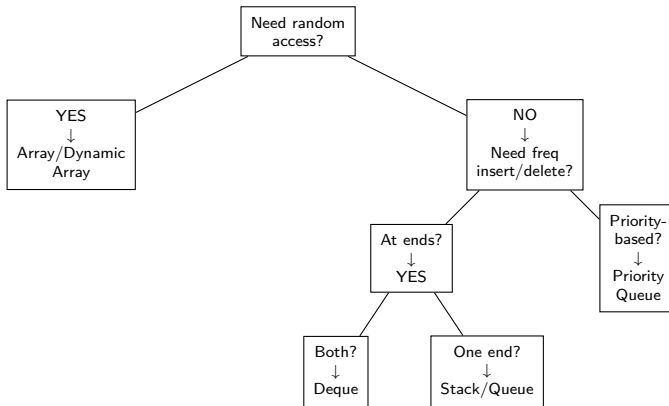
Structure	Access	Insert (begin)	Insert (end)	Delete (begin)	Delete (end)	Search
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Dynamic Array	$O(1)$	$O(n)$	$O(1)^*$	$O(n)$	$O(1)$	$O(n)$
Singly Linked	$O(n)$	$O(1)$	$O(n)^{**}$	$O(1)$	$O(n)$	$O(n)$
Doubly Linked	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Stack	N/A	$O(1)$	N/A	$O(1)$	N/A	N/A
Queue	N/A	N/A	$O(1)$	$O(1)$	N/A	N/A
Deque	N/A	$O(1)$	$O(1)$	$O(1)$	$O(1)$	N/A
Priority Queue	N/A	$O(\log n)$	$O(\log n)$	$O(\log n)$	N/A	$O(n)$

*amortized, ** $O(1)$ with tail pointer

Memory Overhead Comparison

- **Array:** Minimal (just elements) + potential unused capacity
- **Singly Linked List:** 1 pointer per node

Decision Guide: Which Structure to Use?



Real-World Use Cases

Scenario	Structure	Reason
Student grades (by ID)	Dynamic Array	Fast access by index, known size
Text editor undo/redo	Two Stacks	LIFO matches undo/redo behavior
Web server requests	Queue	FIFO ensures fairness
Music playlist	Doubly Linked List	Easy forward/backward, insert anywhere
CPU scheduling	Priority Queue or Circular Queue	Priority-based or round-robin
Browser history	Deque	Fast back/forward navigation

Common Mistakes to Avoid

1. Using `list.pop(0)` for Queue

```
1 # BAD: O(n) - shifts all elements!
2 queue = []
3 queue.append(1)
4 queue.pop(0)
5
6 # GOOD: O(1) with deque
7 from collections import deque
8 queue = deque()
9 queue.append(1)
10 queue.popleft()
```

2. Using Linked List for Random Access

```
1 # BAD: if you need frequent access by index
2 linked_list.get(100) # O(n) traversal
3
4 # GOOD: use array
5 array[100] # O(1)
```

3. Not Considering Cache Performance

Summary

Key Takeaways

Arrays

- $O(1)$ access, cache-friendly, but $O(n)$ insert/delete in middle
- Dynamic arrays: $O(1)$ amortized append via doubling strategy
- Use when: need random access, size known/stable

Linked Lists

- $O(1)$ insert/delete at known positions, dynamic size
- Singly: 1 pointer, forward only; Doubly: 2 pointers, bidirectional
- Use when: frequent insertions/deletions, no random access needed

Stacks & Queues

- Stack (LIFO): function calls, undo, expression parsing, DFS
- Queue (FIFO): task scheduling, BFS, fair processing
- Deque: flexible both ends, palindromes, sliding windows

Implementation Guidelines

From Scratch vs Libraries

Implement from scratch to learn:

- Understand internal mechanisms
- Practice pointer manipulation
- Learn complexity trade-offs

Use library implementations for production:

- Python: `list`, `collections.deque`, `heapq`
- C++: `std::vector`, `std::list`, `std::stack`, `std::queue`
- Java: `ArrayList`, `LinkedList`, `Stack`, `PriorityQueue`

Remember

- Choose based on most frequent operations
- Consider cache performance for large datasets

Thank You!

Questions?

*“The right data structure makes the algorithm simple.
The wrong data structure makes it impossible.”*